```
        void before_each()
        {
            member = new FrequentFlyer();
        }

        void earning_status_points()
        {
            context["When the frequent flyer account is created"] = () =>
            {
                it["should have BRONZE status"] = () =>
                    member.getStatus().should_be(Status.Bronze);

                it["should have 0 status points"] = () =>
                    member.getStatusPoints().should_be(0);
            };

            context["When cumulating Frequent Flyer points"] = () =>
            {
                it["should earn points for each flight"] = () =>
                {
                    member.earnStatusPoints(100);
                    member.earnStatusPoints(50);
                    member.getStatusPoints().should_be(150);
                };

                it["should get upgrade when enough points are earned"] = () =>
                {
                    member.earnStatusPoints(00);

                    member.getStatus().should_be(Status.Silver);
                };
            };
        }
    }
```

A new specification context

Specifications within the context

"should"-style assertions

This is very close, at least in spirit, to the RSpec and Jasmine examples you saw earlier. As with those other tools, you describe your specifications in natural language and group them within contexts, using very clean "should"-style assertions.

10.4.3 Writing more expressive specifications using Spock or Spec2

Tools like RSpec, Jasmine, and NSpec are great ways of expressing BDD-style low-level specifications, but they do have some limitations. For example, there are times when it would be nice to have a more explicit "Given ... When ... Then" structure. Although good practitioners do this with whatever tool they're using, it can often help teams to work more consistently if the structures are more visible.

In addition, example-driven specifications can often be expressed more concisely using examples. You saw the use of table-based examples with tools like JBehave and Cucumber, but they're equally useful at the unit-testing level.

RSpec was released over eight years ago, and more recently a new generation of BDD unit-testing tools has emerged. These newer tools provide the expressiveness of Gherkin, including support for example tables and clear "Given ... When ... Then" structures, without the overhead of maintaining separate feature files and step-definition classes.